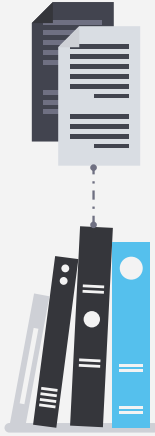


Secure LLM Assistant

COMP.SEC.300
Ville Viinikka

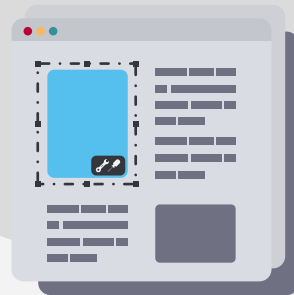
Secure LLM Assistant

- Secure LLM application running on CSC cloud
- React and Nginx frontend with FastAPI backend and Ollama runtime
- Core focus on JWT sessions 2FA and automated DevSecOps pipelines



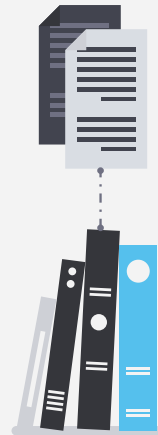
02

Project Goal



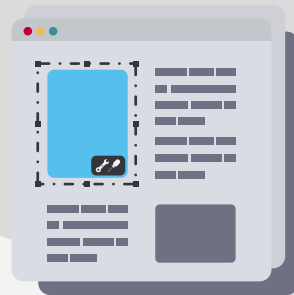
Project Goal

- Build a local LLM assistant safely
- Establish a strict backend security layer in front of the model
- Prevent direct model exposure to the user
- Combine secure coding privacy authentication and DevSecOps



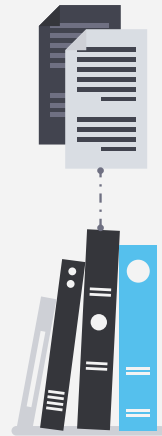
03

Earlier Work



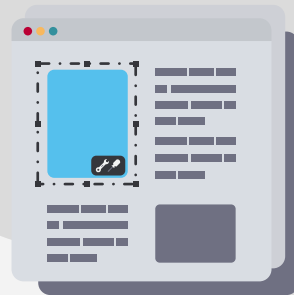
Earlier Work

- Based on a standard local LLM chat architecture but the project is a new one.
- Extended with security features and implementation



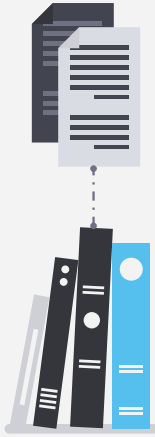
04

Technologies Used



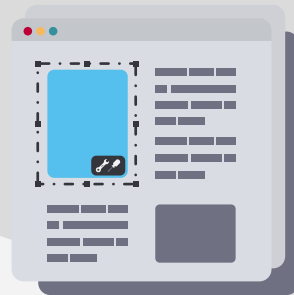
Technologies Used

- Backend: Python, FastAPI, Pydantic, httpx, SQLite, PyJWT, pyotp
- Frontend: JavaScript, React, Vite, Nginx
- LLM runtime: Ollama
- DevOps: Docker Compose, Jenkins, Dependabot
- Testing and security tools: pytest, Bandit, pip-audit, npm audit



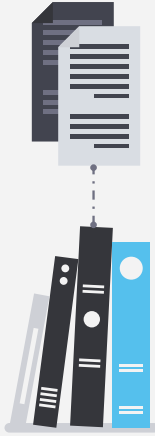
05

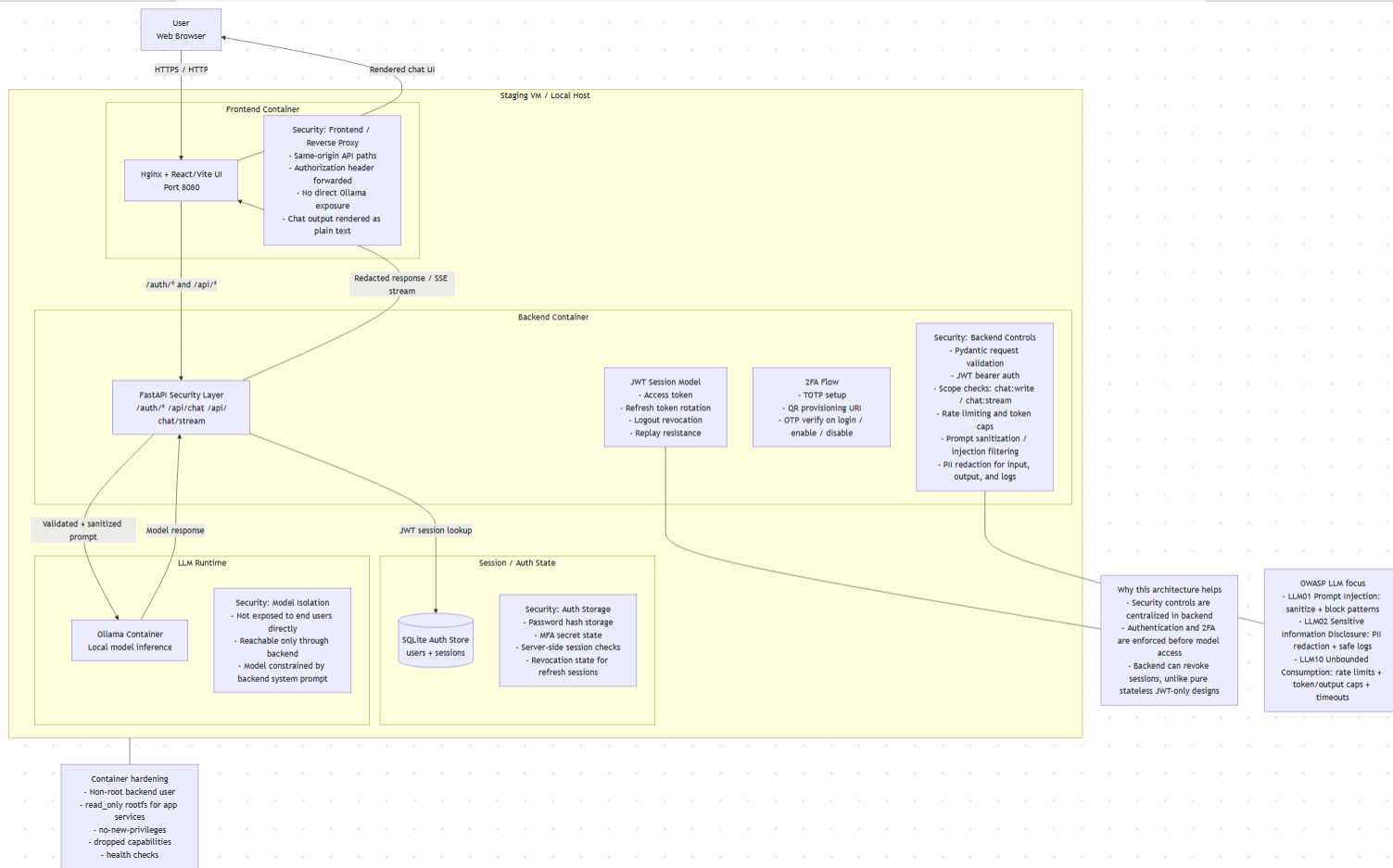
Architecture



Architecture

- User connects to Frontend which routes to Backend which queries Ollama
- Backend serves as the absolute trust boundary
- Validation authentication redaction and limits execute before model access





Why this architecture helps

- Security controls are centralized in backend
- Authentication and 2FA are enforced before model access
- Backend can revoke sessions, unlike pure stateless JWT-only designs

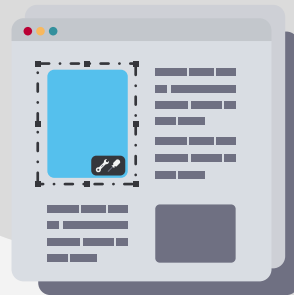
OWASP LLM focus

- LLM01 Prompt Injection: sanitize + block patterns
- LLM02 Sensitive Information Disclosure: PII redaction + safe logs
- LLM10 Unbounded Consumption: rate limits + token/output caps + timeouts



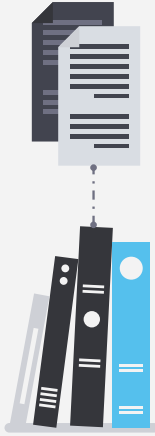
06

Security Controls



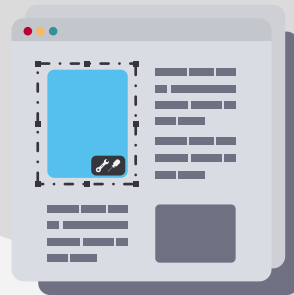
Security Controls

- Prompt sanitization and prompt injection filtering
- PII redaction for prompts responses and application logs
- Request validation using Pydantic models
- Rate limiting token caps and strict timeout handling



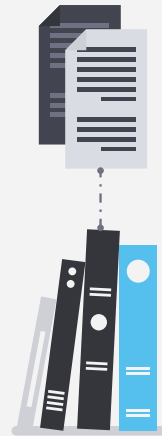
07

JWT Session Database

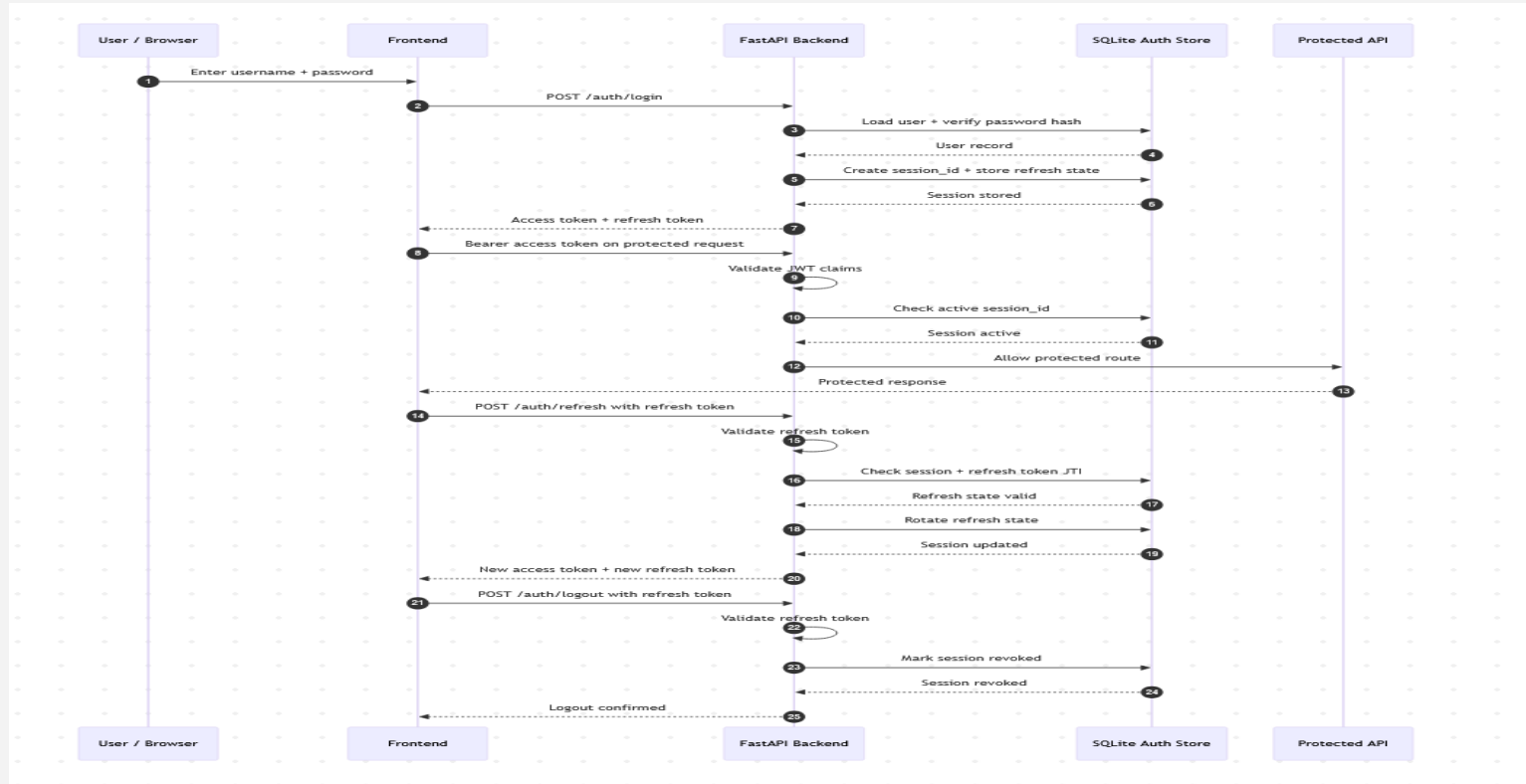


JWT Session Database

- Access token and refresh token model
- SQLite backed server side session store
- Refresh token rotation
- Logout revocation and replay resistance
- Login salted, hashed 240,000 iterations

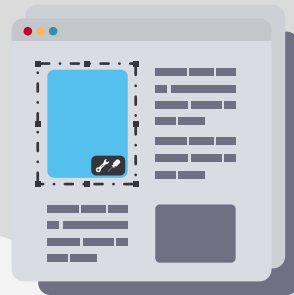


JWT Session Database



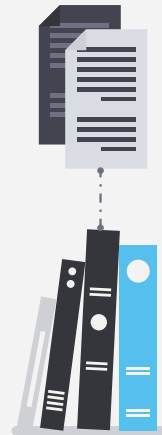
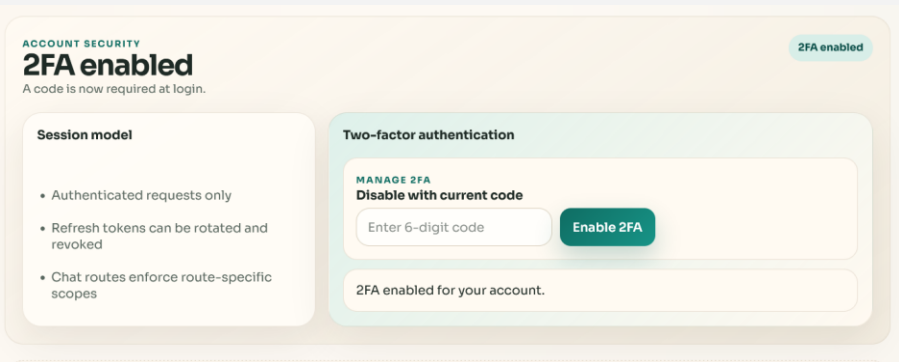
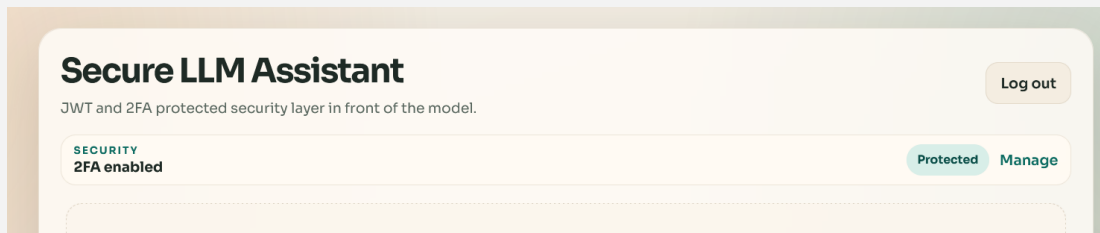
08

2FA Implementation



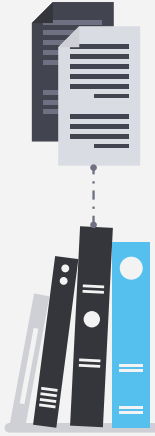
2FA Implementation

- TOTP implementation using pyotp
- OTP required on login when enable



2FA Implementation

- TOTP implementation using pyotp
 - OTP required on login when enable
1. Create user
 2. Start 2FA setup
 3. Scan the QR code
 4. Setup the 2FA on mobile phone
 5. 2FA setup complete



2FA Demo

Secure LLM Assistant

JWT and 2FA protected security layer in front of the model.

Log out

ACCOUNT SECURITY

Setup in progress

Scan and verify your authenticator.

Session model

- Authenticated requests only
- Refresh tokens can be rotated and revoked
- Chat routes enforce route-specific scopes

Two-factor authentication

STEP 1

Scan the QR code

Use any TOTP app.



Open provisioning URI

STEP 2

Confirm setup

Or paste the secret manually.

VV6L7H6RHAJ2C3ZHSUAGANT3PL9MEF

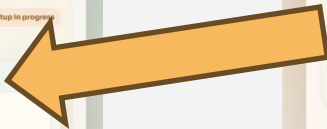
Enter 6-digit code

Enable 2FA

Secret generated. Add it to your authenticator app and verify.

Setup in progress

Ask something...



Secure LLM Assistant

JWT and 2FA protected security layer in front of the model.

Log out

ACCOUNT SECURITY

2FA disabled

Add an authenticator code to your login.

Session model

- Authenticated requests only
- Refresh tokens can be rotated and revoked
- Chat routes enforce route-specific scopes

Two-factor authentication

Start setup

Scan, verify, and 2FA is active.

2FA disabled

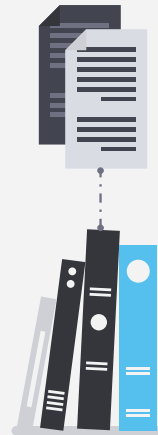
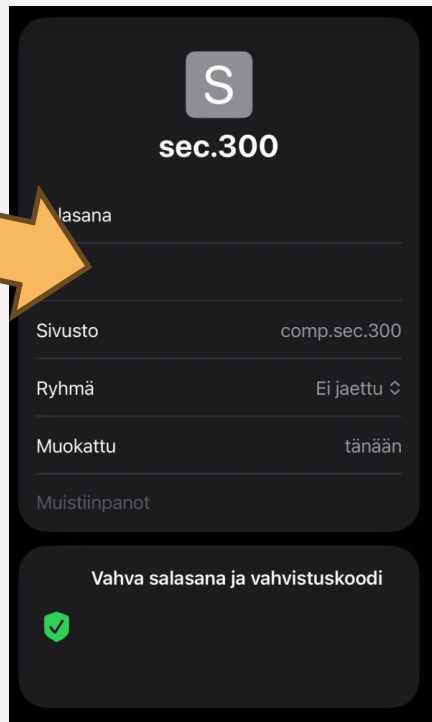
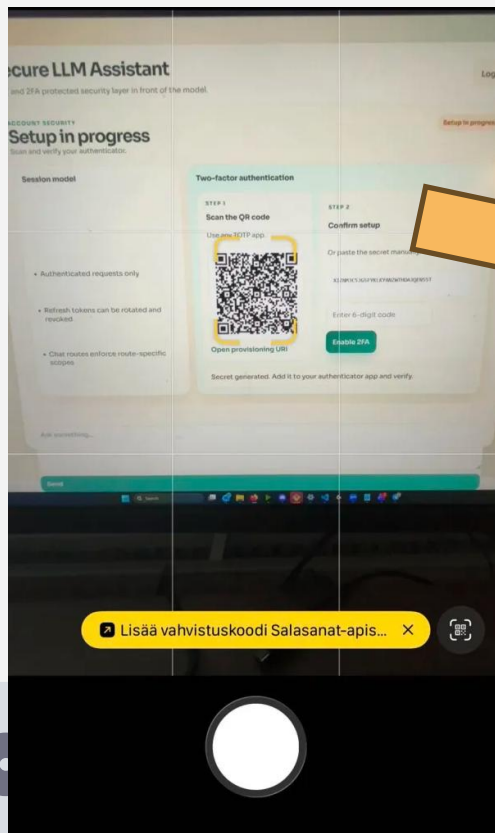
Start the conversation. Press Enter to send, Shift+Enter for newline.

Ask something...

Send



2FA Demo



2FA Demo

Completed
Next time login asks
for 2FA

Secure LLM Assistant

JWT and 2FA protected security layer in front of the model.

PROTECTED ACCESS

Sign in to the secure LLM gateway

JWT

Session-backed tokens
Refresh and logout stay revocable server-side.

2FA

Authenticator app support
Add a second factor to login.

Guardrails

Secure backend first
Checks run before model access.

Login Create account

ACCOUNT ACCESS

Welcome back

Sign in to continue.

Username

Password

Authenticator code Required for this account

Login

Need an account? Create one.

Enter your 6-digit authenticator code.

Secure LLM Assistant

JWT and 2FA protected security layer in front of the model.

Log out

SECURITY

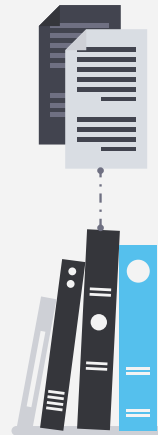
2FA enabled

Protected Manage

Start the conversation. Press Enter to send, Shift+Enter for newline.

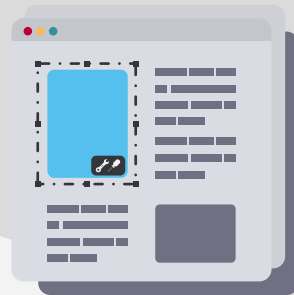
Ask something...

Send



09

Application Demo



Application Demo



Secure LLM Assistant

JWT and 2FA protected security layer in front of the model.

PROTECTED ACCESS

Sign in to the secure LLM gateway

JWT

Session-backed tokens

Refresh and logout stay revocable server-side.

2FA

Authenticator app support

Add a second factor to login.

Security

Secure backend first

Checks run before model access.

Login

Create account

NEW ACCOUNT

Create a protected workspace

Create a total account.

Username

testtesttest1

Password

Use letters and numbers.

Create account

Already have an account? Login instead.

Secure LLM Assistant

JWT and 2FA protected security layer in front of the model.

Log out

SECURITY
2FA enabled

Protected Manage

Start the conversation. Press Enter to send, Shift+Enter for newline.

Ask something...

Send

Secure LLM Assistant

JWT and 2FA protected security layer in front of the model.

Log out

SECURITY
2FA enabled

Protected Manage

You 18.29
Hi, I'm on a course called "Secure Programming" say hello the the course personnel

Assistant 18.29
Hello! I don't have direct access to the specific course personnel or instructors, but I can offer you general support and guidance related to secure programming concepts. How can I assist you with your course?

You 18.30
Tell me about OWASP Mapping

Assistant 18.30
OWASP (Open Web Application Security Project) Mapping is a project that aims to create a comprehensive catalog of known web application security vulnerabilities, threats, and weaknesses.

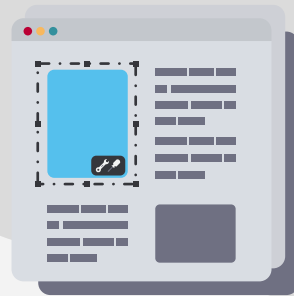
The OWASP Top 10 provides a list of the most critical web application security risks, which serves as a foundation for the OWASP Mapping. The mapping helps

Ask something...

Send

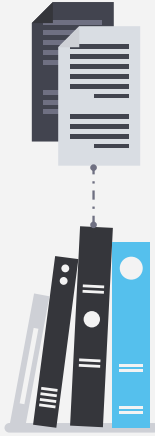
10

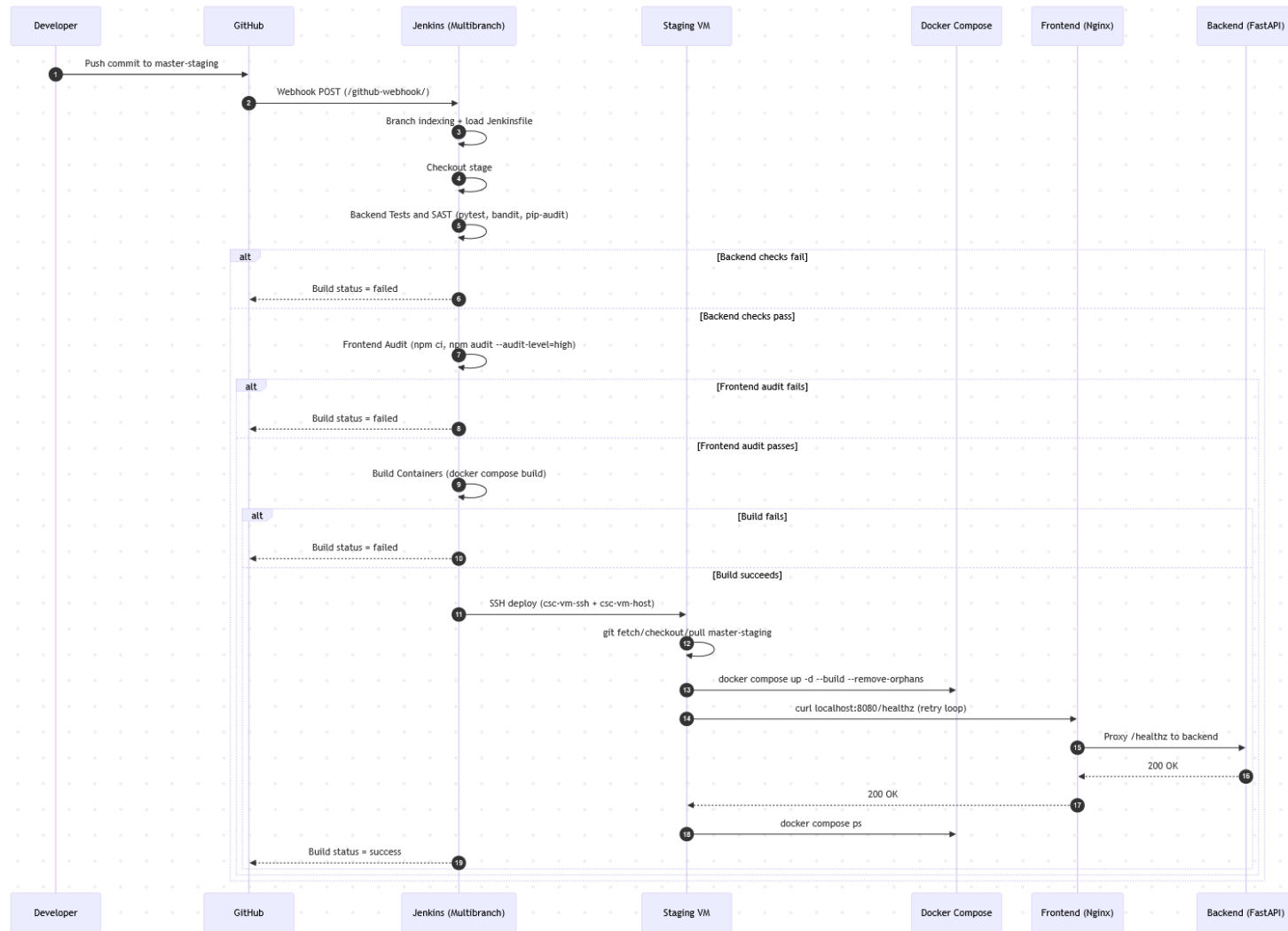
CI CD Pipeline

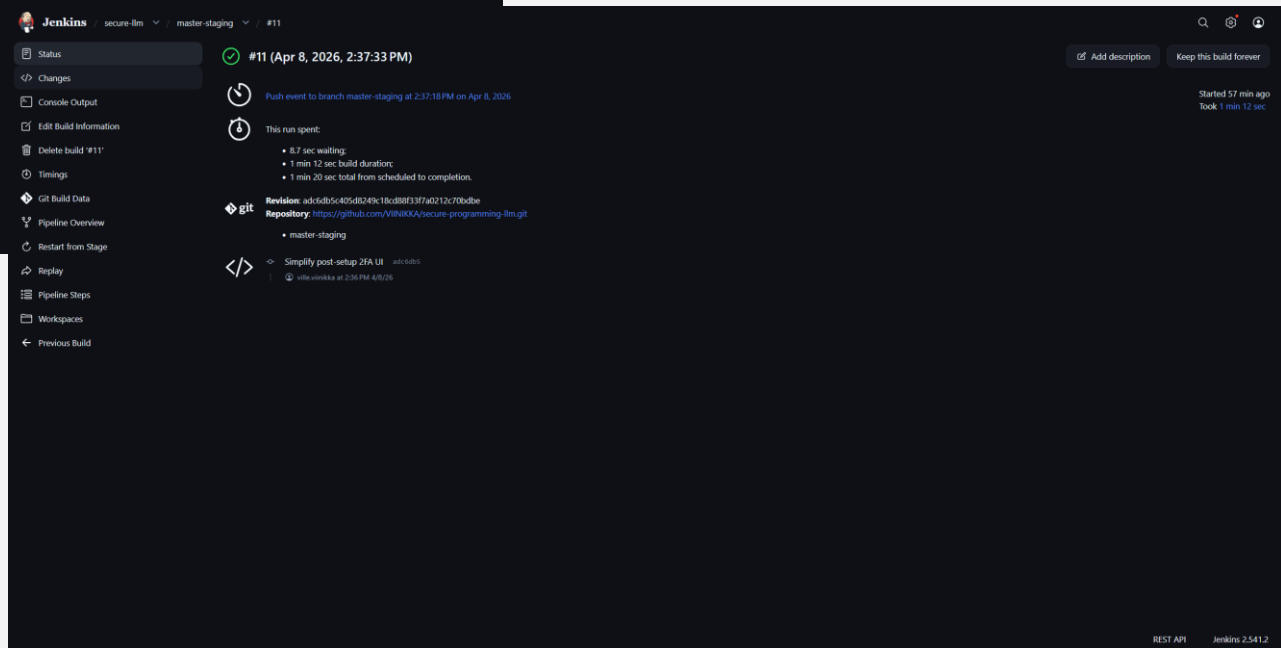
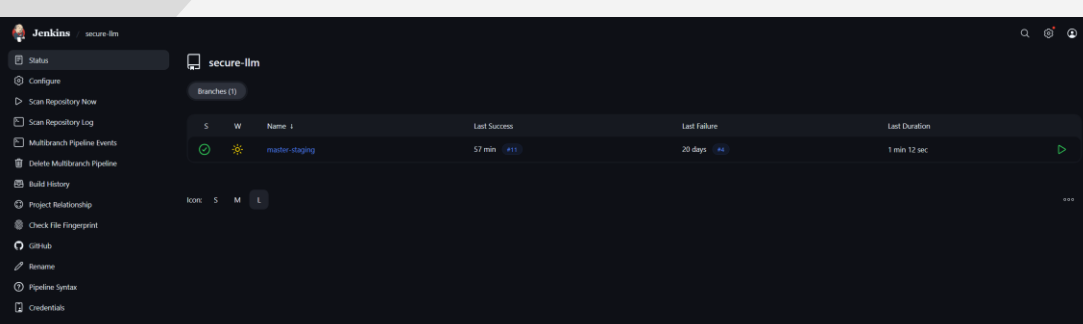


CI CD Pipeline

- Jenkins automated pipeline runs tests and security checks
- Executes pytest bandit pip audit and npm audit
- Automates container build and deployment to a staging environment
- Verifies deployment with an automated health check

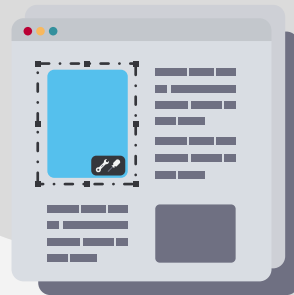






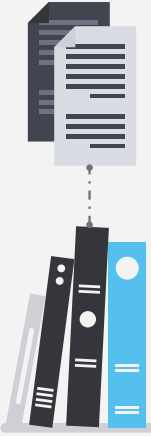
11

AI Usage



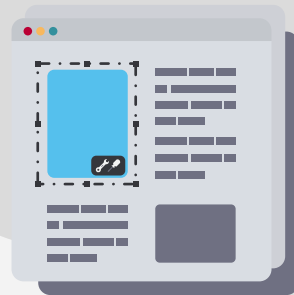
AI Usage

- AI assisted with coding support, documentation improvements, and testing ideas.
- Final design, implementation review, testing, and integration were done manually.
 - AI was used as a tool to learn more from the security domain



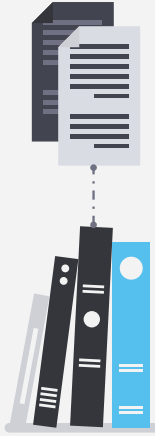
12

What I learned



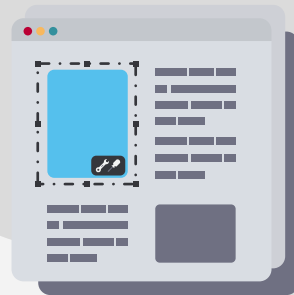
Continue on GitHub

- Backend security architecture
- JWT and session management
- 2FA integration
- Security-focused testing
- CI/CD security checks
- Containerized deployment
- Practical use of AI in development



13

Continue on GitHub



Continue on GitHub

- <https://github.com/VIINIKKA/secure-programming-llm>
- Open these files first
- README.md and SECURITY.md

